

CUDA Streams: Placement and Interview Guide

This guide uses the CUDA Runtime API and C++. It covers the execution model behind asynchronous work, streams, asynchronous copies, events, overlap, concurrent kernels, and synchronization. CUDA operations are often *asynchronous with respect to the host*, but actual concurrency depends on dependencies, memory type, launch configuration, hardware resources, copy engines, and the selected default-stream mode.

Asynchronous Execution

1. Overview

Asynchronous execution means the CPU can submit GPU work and continue without waiting for that work to finish. A CUDA kernel launch is normally asynchronous with respect to the host: the launch call places work into a CUDA command queue and returns after submission, not after every GPU thread finishes.

This matters because CPU waiting time is wasted opportunity. While the GPU processes one batch, the CPU can prepare the next batch, perform I/O, or enqueue more work. Asynchrony is used in inference servers, scientific simulations, image pipelines, databases with GPU operators, and training systems. Interviewers ask about it because it separates launch order, completion order, and true parallel execution—three ideas candidates often confuse.

2. Core Idea

Think of the CPU as a restaurant cashier and the GPU as the kitchen. The cashier writes an order and immediately accepts the next customer. Submission is not completion; a receipt only proves the kitchen received the order.

```
kernel<<<grid, block>>>(d_data); // normally returns before kernel completes
do_cpu_work();                    // may run while the GPU is busy
cudaDeviceSynchronize();          // wait before using the final GPU result
```

Step by step:

1. Host code prepares parameters and device memory.
2. The launch call submits a kernel to a stream.
3. The runtime returns control to the host.
4. CPU and GPU may progress simultaneously.
5. A later dependency or synchronization point makes the host wait if the GPU has not finished.

The word **may** is essential: asynchronous submission enables overlap, but does not guarantee that two operations run simultaneously.

3. Important Subtopics

Subtopic	What it means and why it matters	Example	Common interview angle
Host-asynchronous call	The API can return before device work completes.	Kernel launch.	Asynchronous is relative to the calling host thread.
Device execution order	Operations in one stream execute in issue order.	Kernel B follows kernel A.	Ordered does not mean the host blocks.
Queuing	Work may wait before hardware resources become free.	Ten kernels submitted rapidly.	Enqueued is not running and not completed.
Implicit synchronization	Some API calls force waiting without an explicit synchronize call.	A blocking device-to-host copy.	Hidden barriers can destroy overlap.
Error reporting	Launch errors and execution errors can appear at different times.	Invalid configuration vs illegal memory access.	Check launch status, then a completion boundary.
Host blocking vs device dependency	The CPU can block, or device work can wait on other device work.	<code>cudaDeviceSynchronize</code> vs <code>cudaStreamWaitEvent</code> .	Prefer the narrowest dependency needed.

4. Real-World Example

An inference server receives camera frames. The CPU decodes frame **N+1** while the GPU preprocesses and infers on frame **N**. When implemented as a pipeline, latency stages overlap and throughput rises even if the time for one frame does not shrink.

```

Time --->
CPU:  decode F0 | decode F1 | decode F2 |
GPU:      preprocess+infer F0 | preprocess+infer F1 |

```

5. Diagrams / Mental Models

```

Host thread: submit A -- submit B -- CPU work -- wait -- read result
                |           |
                v           v
GPU stream:    [ queued A ][ queued B ]
GPU engine:    execute A ---- execute B

```

Keep three timestamps separate:

Moment	Meaning
API returned	Host finished submitting the call.
Operation started	GPU engine began executing it.
Operation completed	Results and side effects are complete.

6. Common Interview Questions

1. **What does asynchronous mean in CUDA?** A call may return before the submitted GPU operation completes. Expected: say "with respect to the host." Mistake: claiming all asynchronous operations run concurrently.
2. **Are CUDA kernel launches asynchronous?** Normally yes with respect to the host. Expected: later synchronization or blocking API calls can make completion visible. Mistake: assuming the result is immediately safe to read.
3. **Does asynchronous submission guarantee overlap?** No. Dependencies, stream ordering, hardware engines, and resource pressure determine overlap. Mistake: treating queueing as concurrency.
4. **Why is asynchronous execution useful?** It permits CPU/GPU overlap, batching, and pipelines, improving utilization and throughput. Mistake: promising lower single-operation execution time.
5. **How do you know when asynchronous work is complete?** Query or synchronize a stream/event, synchronize the device, or rely on a defined dependency. Mistake: using host elapsed time after launch.
6. **How are asynchronous errors detected?** Check immediate launch/configuration status with `cudaGetLastError`, then check a synchronization boundary for execution failures. Mistake: assuming the launch return captures a later illegal access.
7. **What is implicit synchronization?** An API action introduces a wait or global ordering even though no explicit synchronization function appears. Mistake: assuming every runtime call is nonblocking.
8. **Can the CPU read device memory directly after launch?** Usually no; it must transfer or access supported mapped/unified memory with correct synchronization. Mistake: confusing address availability with completion.
9. **What is the difference between blocking and asynchronous copies?** A blocking copy may hold the host until the copy completes; an async copy is enqueued into a stream and returns earlier when its requirements are met. Mistake: ignoring pinned-memory requirements.
10. **Can multiple host threads enqueue GPU work?** Yes, subject to CUDA context and stream rules. Expected: distinguish thread safety from stream ordering. Mistake: assuming host threads automatically create independent device execution.

7. Deep-Dive Questions

1. **Why might an asynchronous API call still block briefly?** Runtime initialization, command-buffer pressure, memory staging, allocation, or implementation details can require host work. The programming model does not promise zero host overhead.
2. **How do you separate launch errors from execution errors?** Call `cudaGetLastError()` after launch for configuration errors, then inspect the status returned by a suitable completion operation for runtime execution errors.
3. **Can CPU work always overlap a kernel?** It can if the launch returns and the host does independent work, but CPU scheduling, synchronous APIs, page faults, and dependencies may limit useful overlap.

4. **Why can excessive asynchronous submission hurt?** It can increase queue depth, memory lifetime, latency, and backpressure. Bounded pipelines make resource ownership clearer.
5. **How does Unified Memory affect the picture?** Page migration and faults can introduce data movement and stalls. Prefetching and correct synchronization may make behavior more predictable.

8. Comparison Tables

Synchronous execution	Asynchronous execution
Caller waits for completion	Caller may continue after submission
Simple lifetime reasoning	Requires explicit dependency reasoning
Limited host/device overlap	Enables pipelines and overlap
Timing around call may measure work	Timing around call may measure only launch overhead
Submission	Concurrency
Describes when the host call returns	Describes simultaneous execution
Primarily an API semantic	Depends on hardware and dependencies
Can occur without overlap	Requires independent work and resources

9. Common Mistakes

- Equating “asynchronous” with “parallel.”
- Reading or freeing a buffer before queued work using it completes.
- Measuring a kernel with a CPU timer but forgetting to synchronize.
- Calling `cudaDeviceSynchronize()` after every launch and eliminating the benefit.
- Checking only the kernel launch and missing delayed execution errors.
- Assuming all CUDA API calls have identical synchronization behavior.

10. Edge Cases / Special Cases

- The first CUDA call may include context initialization overhead.
- Pageable host memory may force staging and host blocking for nominally asynchronous transfers.
- Some memory management and legacy default-stream operations can introduce broad ordering.
- A launch can be asynchronous even if the GPU later serializes it behind earlier work.
- Profilers can perturb timing; warm up and measure repeated steady-state work.

11. How to Explain in Interview

“Asynchronous CUDA execution means the host submits work and can continue before the GPU completes it. It enables CPU/GPU pipelines, but does not guarantee concurrent device execution. I use streams and events to express ordering, and synchronize only where data ownership or result visibility requires it.”

12. Quick Revision Notes

- Kernel launch: normally host-asynchronous.
- Submission is not start or completion.

- Same-stream work remains ordered.
- Overlap needs independence and hardware support.
- Use CUDA events for GPU timing.
- Check immediate and delayed errors.
- Interview trap: async does not mean concurrent.

13. Practice Tasks

1. Launch a vector-add kernel, perform independent CPU work, then synchronize and verify the result.
2. Time a kernel with and without a synchronization before stopping a CPU timer; explain the difference.
3. Add `cudaGetLastError()` and completion-error checking to a kernel program.
4. Draw submission, start, and completion timelines for three queued kernels.
5. Find every unnecessary `cudaDeviceSynchronize()` in a sample loop and replace it with the narrowest safe dependency.

14. Final Cheat Sheet

Item	Must remember
Core definition	Host returns before submitted device work finishes.
Why it matters	Enables CPU/GPU overlap and pipelining.
Most asked	Is async the same as concurrent? No.
Key comparison	Submission semantics vs actual hardware overlap.
Main trap	Timing a launch without waiting for completion.
One-line answer	"Async CUDA calls enqueue work; dependencies and hardware decide when it actually runs."

CUDA Streams

1. Overview

A **CUDA stream** is an ordered sequence of GPU operations. Operations issued to the same stream execute in issue order; operations in different streams may execute concurrently when dependencies and hardware allow.

Streams matter because they let applications express independent pipelines without manually scheduling GPU engines. They are used for chunked data processing, multi-request inference, video processing, numerical solvers, and library integration. Interviewers use streams to test whether you understand ordering scope, default-stream behavior, and the difference between logical independence and physical concurrency.

2. Core Idea

Imagine one checkout line per stream. Customers within a line are served in order. Two lines may be served simultaneously if two cashiers are available; if only one cashier is available, the lines still remain logically separate but take turns.

```

cudaStream_t s;
cudaStreamCreate(&s);

kernelA<<<grid, block, 0, s>>>(d_x);
kernelB<<<grid, block, 0, s>>>(d_x); // begins after A in this stream

cudaStreamSynchronize(s);
cudaStreamDestroy(s);

```

Step by step: create a stream; associate copies and launches with it; CUDA preserves order within it; independent streams become candidates for overlap; synchronize before the host consumes results or reuses resources.

3. Important Subtopics

Subtopic	Meaning and importance	Example	Common interview angle
In-order semantics	Work in one stream observes issue order.	Copy H2D, kernel, copy D2H.	No extra barrier is needed between these same-stream operations.
Inter-stream independence	Different streams have no general ordering unless a dependency is added.	Two input chunks.	Race if both access the same buffer unsafely.
Default stream	Stream used when no stream is specified. Semantics depend on legacy vs per-thread mode.	<code>kernel<<<g,b>>>()</code> ;	Legacy default stream can synchronize broadly.
Nonblocking streams	<code>cudaStreamNonBlocking</code> avoids implicit synchronization with the legacy default stream.	Independent library pipeline.	Name does not mean every API call is host-nonblocking.
Stream priorities	Hint which queued work should be scheduled first.	Latency-sensitive inference stream.	Priority is not preemption or a completion guarantee.
Stream lifecycle	Create, use, synchronize as necessary, destroy.	Long-lived stream pool.	Avoid creating streams per tiny operation.
Per-thread default stream	Each host thread has its own regular default stream when enabled.	Multi-threaded submitters.	Compile/runtime configuration matters.

4. Real-World Example

A batch service divides a large input into four chunks. Each stream performs H2D copy, kernel, and D2H copy for one chunk. Same-stream ordering builds each chunk's pipeline, while different streams let copy engines and SMs work on different chunks.

```
for (int i = 0; i < chunks; ++i) {
    cudaMemcpyAsync(d[i], h_in[i], bytes, cudaMemcpyHostToDevice, s[i]);
    transform<<<grid, block, 0, s[i]>>>(d[i]);
    cudaMemcpyAsync(h_out[i], d[i], bytes, cudaMemcpyDeviceToHost, s[i]);
}
```

5. Diagrams / Mental Models

```
Stream 0: [H2D chunk 0] -> [Kernel 0] -> [D2H chunk 0]
Stream 1: [H2D chunk 1] -> [Kernel 1] -> [D2H chunk 1]
Stream 2: [H2D chunk 2] -> [Kernel 2] -> [D2H chunk 2]
```

Rule: arrows inside a stream are guaranteed ordering.
vertical overlap across streams is possible, not guaranteed.

6. Common Interview Questions

1. **What is a CUDA stream?** An ordered sequence of device operations. Expected: same-stream ordering and possible cross-stream concurrency. Mistake: calling it a CPU thread.
2. **Do operations in one stream overlap each other?** They execute in order; ordinary dependent operations in that stream do not pass one another. Mistake: assuming "stream" itself means parallel execution.
3. **Can operations in different streams run concurrently?** Possibly, if they are independent and hardware/resources allow. Mistake: saying always.
4. **Why does same-stream H2D → kernel → D2H work without explicit barriers?** Stream ordering establishes the required sequence. Mistake: inserting device-wide synchronization after every step.
5. **What is the default stream?** The stream selected when none is specified. Expected: distinguish legacy and per-thread default semantics. Mistake: assuming it is always an ordinary independent stream.
6. **What are legacy default-stream semantics?** The null stream can impose ordering with blocking streams across the context. Mistake: overlooking accidental serialization.
7. **What is a nonblocking stream?** A stream created with `cudaStreamNonBlocking`; it does not implicitly synchronize with the legacy default stream. Mistake: interpreting the flag as "host never blocks."
8. **What are stream priorities?** Scheduling hints favoring higher-priority queued work. Mistake: claiming guaranteed preemption of a running kernel.
9. **How many streams should an application create?** Enough to expose useful independent work, measured on the target workload; more streams add overhead and do not create hardware resources. Mistake: one stream per element.
10. **When can a stream be destroyed?** Destruction releases the handle after queued work is handled according to API semantics; application resources used by that work must remain valid until completion. Mistake: freeing buffers just because the handle was destroyed.

7. Deep-Dive Questions

1. **Can two streams safely write the same allocation?** Only if accesses are disjoint or explicitly ordered; otherwise there is a data race. Stream separation is not synchronization.
2. **How do libraries interact with streams?** Many CUDA libraries store a stream in a library handle. Set the intended stream and understand whether a call uses internal work or the default stream.
3. **Do stream priorities guarantee low latency?** No. They influence selection of pending work, while already-running kernels and unavailable resources may delay high-priority work.
4. **Why might two kernels in different streams serialize?** One may consume all SM resources, an implicit barrier may exist, hardware may lack capacity, or a dependency may connect them.
5. **What are CUDA Graphs' relationship to streams?** Streams express order during capture or direct submission; graphs package a dependency graph to reduce repeated launch overhead. Graphs do not remove data dependencies.

8. Comparison Tables

Same stream	Different streams
Issue order is preserved	No general order without dependency
Easy producer-consumer chain	Enables possible overlap
Shared data naturally sequenced	Shared data can race
Less scheduling flexibility	More scheduling flexibility and complexity
Legacy default stream	Per-thread default stream
Can synchronize with other blocking streams	Behaves like a regular stream per host thread
Shared context-wide null stream behavior	Separate default stream for each host thread
Can accidentally serialize pipelines	Better independence for multi-threaded submission
Controlled by build/runtime mode	Must be enabled consistently

9. Common Mistakes

- Treating a stream as a hardware engine or CPU thread.
- Assuming different streams guarantee simultaneous execution.
- Reusing a host or device buffer while a stream still accesses it.
- Mixing the legacy default stream into a pipeline and causing serialization.
- Using one event or buffer across streams without deliberate ownership rules.
- Creating many short-lived streams instead of reusing a small pool.

10. Edge Cases / Special Cases

- Default-stream behavior can differ across compilation units if configuration is inconsistent.
- Stream priorities are hints and normally do not preempt already-running blocks.
- A device may overlap copies and kernels but still serialize kernels due to occupancy.
- Stream callbacks/host functions must follow CUDA API restrictions and should remain short.
- Separate processes or contexts introduce scheduling concerns beyond ordinary streams.

11. How to Explain in Interview

"A CUDA stream is an in-order queue of device work. Same-stream operations are automatically ordered, while different streams can overlap if I avoid data dependencies and the GPU has resources. I use events for cross-stream dependencies and avoid broad device synchronization."

12. Quick Revision Notes

- Same stream: ordered.
- Different streams: potentially concurrent, not automatically safe.
- Stream is a logical queue, not a dedicated engine.
- Default stream has legacy/per-thread modes.
- Priorities are hints, not preemption guarantees.
- Interview trap: more streams do not mean more concurrency.

13. Practice Tasks

1. Put H2D, kernel, and D2H operations into one nondefault stream and verify ordering.
2. Split an array across two streams and compare the result and timeline with one stream.
3. Create an intentional race by sharing an output buffer, then repair it with disjoint ranges or an event.
4. Compare legacy and per-thread default-stream behavior in a small multi-threaded program.
5. Use Nsight Systems to identify whether operations from separate streams actually overlap.

14. Final Cheat Sheet

Item	Must remember
Core definition	An ordered sequence of CUDA operations.
Why it matters	Expresses pipelines and exposes independent work.
Most asked	Same-stream ordering vs cross-stream concurrency.
Key comparison	Legacy default vs per-thread default stream.
Main trap	Different streams permit but do not guarantee overlap.
One-line answer	"Streams preserve local order and expose cross-stream concurrency."

cudaMemcpyAsync

1. Overview

`cudaMemcpyAsync` enqueues a memory transfer into a CUDA stream and can return before that transfer completes. It supports host-to-device (H2D), device-to-host (D2H), device-to-device (D2D), and other valid transfer directions.

It matters because data movement across PCIe or NVLink is often a major GPU bottleneck. Asynchronous copies let transfers participate in a pipeline and potentially overlap with CPU work, kernels, or transfers on

other copy engines. Interviewers ask about it to test pinned host memory, buffer lifetime, stream ordering, and the difference between an asynchronous API name and actual overlap.

2. Core Idea

Think of a DMA copy engine as a delivery truck. `cudaMemcpyAsync` schedules a delivery in a stream; the host does not need to ride in the truck. For direct, reliably asynchronous host transfers, the source or destination host buffer should be page-locked (pinned), so the operating system cannot move its physical pages during DMA.

```
float *h, *d;
cudaMallocHost(&h, bytes);           // pinned host memory
cudaMalloc(&d, bytes);
cudaStream_t s;
cudaStreamCreate(&s);

cudaMemcpyAsync(d, h, bytes, cudaMemcpyHostToDevice, s);
kernel<<<grid, block, 0, s>>>(d);    // ordered after the copy
cudaStreamSynchronize(s);            // h and d are now safe to reuse here
```

Step by step: allocate suitable memory; enqueue the copy; the stream preserves its order relative to nearby work; hardware moves bytes when dependencies and engines permit; synchronize before the application reuses or consumes the affected storage.

3. Important Subtopics

Subtopic	What it means and why it matters	Example	Common interview angle
Pinned host memory	Physical pages are locked and DMA-addressable.	<code>cudaMallocHost</code> , <code>cudaHostAlloc</code> .	Needed for dependable host/device async overlap.
Pageable host memory	Normal <code>malloc</code> memory can require staging through a pinned buffer.	<code>std::vector<float></code> .	The call may block or overlap poorly.
Copy direction	Direction controls source/destination interpretation.	<code>cudaMemcpyHostToDevice</code> .	<code>cudaMemcpyDefault</code> needs UVA-supported pointer inference.
Stream association	Copy is ordered with other operations in its stream.	H2D → kernel → D2H.	Same-stream sequence needs no intermediate host barrier.
Buffer lifetime	Source and destination must remain valid and unmodified as required until completion.	Double buffers.	Async call returning does not transfer ownership back.

Subtopic	What it means and why it matters	Example	Common interview angle
Copy engines	DMA engines can operate independently of SMs.	H2D while kernel runs.	Number and direction support are device-dependent.
2D/3D async copies	Pitch-aware copies handle arrays and subregions.	<code>cudaMemcpy2DAsync</code> .	Width is in bytes and pitch must be respected.

4. Real-World Example

A video analytics system uses two pinned host buffers. While the GPU processes frame 0 from buffer A, a second stream transfers frame 1 from buffer B. The camera writes only to a buffer whose previous transfer has completed. This double-buffering avoids a race and hides part of the transfer time.

```
Buffer A: fill F0 -> H2D F0 -> reusable -> fill F2
Buffer B:          fill F1 -> H2D F1 -> reusable
GPU:                process F0 -> process F1
```

5. Diagrams / Mental Models

```
Pageable host memory:
application pages -> runtime pinned staging buffer -> DMA -> device
                    possible host-side wait/copy

Pinned host memory:
application pinned pages -----> DMA -> device
```

Pinned memory is a performance tool with a cost: too much reduces the OS's ability to page memory and may degrade system performance.

6. Common Interview Questions

- What does `cudaMemcpyAsync` do?** It enqueues a copy in a stream and may return before completion. Expected: stream ordering and lifetime. Mistake: saying data is ready when the call returns.
- Why is pinned host memory important?** DMA can access stable physical pages directly, enabling reliable asynchronous H2D/D2H transfers and overlap. Mistake: saying pinned memory is GPU memory.
- Will `cudaMemcpyAsync` always be asynchronous?** No; memory type, direction, runtime behavior, and resource conditions matter. Mistake: trusting the function name as a universal guarantee.
- Can a copy and kernel overlap in the same stream?** Their order is serialized in that stream. Use separate streams and independent data for overlap. Mistake: assuming separate engines override stream order.
- Can H2D and D2H copies overlap?** On devices with suitable independent copy engines, possibly. Verify device capability and workload behavior. Mistake: assuming every GPU has two full-duplex engines.

6. **When may the host modify an H2D source buffer?** After the queued transfer using it completes. Mistake: modifying it immediately after the async call returns.
7. **When may the host read a D2H destination?** After the copy completes, established with stream/event/device synchronization or a correct higher-level dependency. Mistake: polling the values without synchronization.
8. **What is the difference between `cudaMemcpy` and `cudaMemcpyAsync`?** The latter is stream-ordered and intended for asynchronous pipelines; exact blocking behavior varies by transfer type. Mistake: reducing the distinction to "fast vs slow."
9. **How do you allocate pinned memory?** `cudaMallocHost`, `cudaHostAlloc`, or register suitable existing memory with `cudaHostRegister`. Mistake: pinning arbitrary short-lived memory without unregistering it.
10. **What limits transfer throughput?** Link bandwidth, transfer size, engine count, memory bandwidth, NUMA placement, and protocol overhead. Mistake: benchmarking only tiny copies.

7. Deep-Dive Questions

1. **Why are many small async copies inefficient?** Fixed submission and protocol overhead dominate. Batch adjacent data or use larger chunks when latency constraints allow.
2. **Can device-to-device copies overlap kernels?** Potentially, depending on the transfer path, engine availability, dependencies, and memory-bandwidth contention.
3. **What happens if source and destination regions overlap?** Do not assume `memcpy` semantics; use a safe transformation or temporary storage when overlap is possible.
4. **How does NUMA placement affect pinned transfer speed?** A CPU socket remote from the GPU may feed pinned pages over an inter-socket link, reducing bandwidth. Allocate/touch buffers near the GPU's NUMA node when relevant.
5. **When is mapped pinned memory useful?** For small or one-pass host data where avoiding an explicit copy helps. Repeated GPU access over PCIe is usually much slower than device memory.

8. Comparison Tables

Pageable host memory	Pinned host memory
Easy, cheap allocation	Limited resource; costlier allocation
OS may page or relocate it	Physical pages remain resident
Runtime may stage transfers	Direct DMA-friendly transfers
Async overlap may be limited	Required for robust copy/compute overlap

<code>cudaMemcpy</code>	<code>cudaMemcpyAsync</code>
Simpler blocking-style use	Enqueued in a specified stream
Often used at phase boundaries	Used in pipelines
Broad host wait behavior depending on transfer	Host can continue when requirements are satisfied
Harder to overlap	Enables possible overlap

9. Common Mistakes

- Using pageable host buffers and expecting full copy/compute overlap.
- Reusing or freeing buffers before transfer completion.
- Putting copy and kernel in one stream while expecting them to overlap.
- Pinning huge amounts of system memory.
- Measuring first-use initialization instead of steady-state bandwidth.
- Forgetting byte units in copy sizes and pitched APIs.

10. Edge Cases / Special Cases

- Zero-byte copies are valid no-ops in many situations but do not substitute for synchronization.
- Registration requires suitable address alignment/range handling and must be paired with unregistering.
- Unified Memory uses migration/prefetch semantics rather than ordinary host/device copy reasoning.
- Peer-to-peer copies require topology and peer-access support for the best path.
- Multiple transfers may contend for PCIe, NVLink, memory controllers, or the same copy engine.

11. How to Explain in Interview

"`cudaMemcpyAsync` enqueues a stream-ordered transfer. For real H2D or D2H overlap I use pinned host memory, keep buffers alive until an event or stream reports completion, and place independent chunks in separate streams. Actual overlap still depends on copy engines and bandwidth."

12. Quick Revision Notes

- Async copy is enqueued, not instantly complete.
- Pinned host memory is the standard requirement for overlap.
- Same stream preserves H2D → kernel → D2H order.
- Separate streams expose overlap.
- Buffer lifetime lasts through completion.
- Too much pinned memory harms the host.
- Interview trap: `Async` in the name is not an unconditional guarantee.

13. Practice Tasks

1. Measure pageable versus pinned H2D bandwidth for several transfer sizes.
2. Build a two-buffer pipeline with one event per reusable host buffer.
3. Put copy and compute in one stream, then separate streams, and inspect both timelines.
4. Implement a pitched 2D image copy and verify row addressing.
5. Query device properties and relate copy-engine capability to profiler results.

14. Final Cheat Sheet

Item	Must remember
Core definition	Enqueues a memory copy in a stream.
Why it matters	Enables transfer pipelines and possible overlap.
Most asked	Why pinned memory? Stable DMA-accessible pages.
Key comparison	Pageable staging vs pinned direct DMA path.

Item	Must remember
Main trap	Reusing a buffer before its copy completes.
One-line answer	"Async copies need correct memory, stream, lifetime, and hardware conditions to overlap."

CUDA Events

1. Overview

A **CUDA event** is a marker recorded in a stream. It becomes complete after all earlier operations in that stream reach the event. Host code can query or wait for it, and another stream can wait on it without blocking the host.

Events matter for fine-grained dependencies, completion tracking, buffer reuse, and GPU-side timing. They appear in pipelines, stream coordination, benchmarks, and producer-consumer scheduling. Interviewers ask about events because they are lighter and more precise than frequent device-wide synchronization.

2. Core Idea

An event is like placing a checkpoint card at a position in a checkout line. The card is stamped only after every customer before it has been served. Another line can be told, "do not serve this customer until that card is stamped."

```
cudaEvent_t ready;
cudaEventCreate(&ready);

producer<<<g, b, 0, producerStream>>>(d_data);
cudaEventRecord(ready, producerStream);
cudaStreamWaitEvent(consumerStream, ready, 0);
consumer<<<g, b, 0, consumerStream>>>(d_data);

cudaEventDestroy(ready);
```

The host submits this dependency and can continue. The consumer stream waits on the device only where required.

3. Important Subtopics

Subtopic	Meaning and importance	Example	Common interview angle
Event recording	Inserts an event marker into a stream.	<code>cudaEventRecord(e, s).</code>	Completion covers earlier work in that stream.

Subtopic	Meaning and importance	Example	Common interview angle
Event query	Nonblocking host check.	<code>cudaEventQuery(e).</code>	<code>cudaErrorNotReady</code> is a state, not a fatal failure.
Event synchronization	Blocks host until event completion.	<code>cudaEventSynchronize(e).</code>	Narrower than device-wide synchronization.
Stream wait	Makes future work in one stream wait for an event.	<code>cudaStreamWaitEvent.</code>	Device-side dependency; host need not wait.
Timing events	Timestamped markers measure GPU elapsed time.	<code>cudaEventElapsedTime.</code>	Both events must be recorded appropriately and completed.
Event flags	Disable timing or request blocking host waits.	<code>cudaEventDisableTiming.</code>	Disable timing for lower-overhead dependency-only events.
Re-recording	Recording an event again associates it with the newest record.	Reused ring-buffer event.	Avoid ambiguous lifetime/iteration logic.

4. Real-World Example

In a multi-stage image pipeline, stream A preprocesses an image, records `preprocessDone`, and stream B waits for it before inference. Stream C, handling an independent image, remains unaffected. An event encodes exactly one dependency instead of freezing the entire device.

5. Diagrams / Mental Models

```

Producer stream: [H2D] -> [preprocess] -> (event E)
                                   |
                                   v
Consumer stream: [independent work] ---- wait(E) -> [inference]

Host: record E -> submit wait(E) -> continue immediately

```

For timing:

```

stream: record(start) -> kernel(s) -> record(stop)
elapsed = stop GPU timestamp - start GPU timestamp

```

6. Common Interview Questions

1. **What is a CUDA event?** A stream-recorded completion marker usable for queries, waits, and timing.
Mistake: describing it as a kernel or OS thread event only.
2. **When does a recorded event complete?** After all earlier work in the recording stream completes up to that marker. Mistake: saying immediately when `cudaEventRecord` returns.
3. **How do you synchronize two streams?** Record an event in the producer, call `cudaStreamWaitEvent` in the consumer, then enqueue consumer work. Mistake: blocking the entire device.
4. **Does `cudaStreamWaitEvent` block the CPU?** No; it inserts a dependency into the waiting stream.
Mistake: confusing it with `cudaEventSynchronize`.
5. **How do you time a kernel accurately?** Record start and stop events in a suitable stream, synchronize the stop event, and call `cudaEventElapsedTime`. Mistake: CPU timing an asynchronous launch without a completion wait.
6. **What does `cudaEventQuery` return before completion?** `cudaErrorNotReady`. Expected: treat it as a pending status. Mistake: reporting it as program failure.
7. **Why use `cudaEventDisableTiming`?** Dependency-only events need no timestamp, reducing overhead. Mistake: later attempting elapsed-time measurement with such events.
8. **Event synchronization vs stream synchronization?** Event synchronization waits through one marker; stream synchronization waits for all previously submitted work in that stream. Mistake: calling them globally equivalent.
9. **Can an event be reused?** Yes, but re-recording changes the completion point represented by subsequent operations; manage iterations carefully. Mistake: overwriting a marker still needed to distinguish old work.
10. **Do events provide data visibility/order?** A stream wait on a completed producer event establishes execution order for subsequent consumer work. Mistake: querying an event and then assuming unrelated already-enqueued work was reordered.

7. Deep-Dive Questions

1. **Why are events better than host synchronization for device pipelines?** They express dependencies on the device, allowing the host and unrelated streams to continue.
2. **Can event timing include work from other streams?** Elapsed time reflects timestamp positions; scheduling interference from other work can increase the measured interval even when only one stream's markers are used.
3. **What if an event is recorded after a wait on the same event?** Careless dependency cycles or reliance on a future re-record can deadlock or create incorrect logic. Dependencies must form an acyclic graph for each iteration.
4. **How accurate are event timings?** They are appropriate GPU timeline measurements with finite resolution; warm-up, repetitions, clock behavior, and interference still matter.
5. **Can events coordinate devices?** Cross-device behavior has API-specific restrictions. For multi-GPU designs, verify event, peer-access, IPC, and synchronization support rather than assuming ordinary same-device semantics.

8. Comparison Tables

CUDA event	CPU wall-clock timer
Positioned in GPU stream	Measured on host
Handles asynchronous completion naturally	Requires explicit completion before stop time

CUDA event	CPU wall-clock timer
Good for kernel/copy elapsed time	Good for end-to-end application latency
Excludes or includes queue delay based on marker placement	Includes host overhead and waits around measured region
<code>cudaEventSynchronize</code>	<code>cudaStreamWaitEvent</code>
Blocks the calling host thread	Enqueues a wait in a device stream
Host needs completion now	Device consumer needs producer ordering
Narrow host-side wait	Fine-grained cross-stream dependency

9. Common Mistakes

- Recording timing events in the wrong stream.
- Reading elapsed time before the stop event completes.
- Replacing every dependency with `cudaDeviceSynchronize()`.
- Treating `cudaErrorNotReady` as fatal.
- Re-recording one event while old iterations still rely on its identity.
- Timing first-run initialization or JIT overhead as steady-state kernel time.

10. Edge Cases / Special Cases

- An event recorded in an idle stream can complete quickly but record is still an enqueued operation.
- Event timing may include scheduling delay between markers.
- Busy-wait vs blocking-sync behavior can be influenced by event flags and runtime scheduling settings.
- Destroying an event handle does not make earlier dependent GPU work safe to ignore; resource lifetimes still matter.
- Interprocess events require explicit creation/export mechanisms and have restrictions.

11. How to Explain in Interview

"A CUDA event is a marker recorded in a stream that completes after earlier stream work. I use it for GPU timing, host completion checks, and especially `cudaStreamWaitEvent` to connect producer and consumer streams without synchronizing the whole device."

12. Quick Revision Notes

- Record event into a stream.
- Event completes after prior stream work.
- Query: nonblocking host check.
- Synchronize event: host waits.
- Stream wait event: device dependency; host continues.
- Timing events use GPU timestamps.
- Interview trap: record returning does not mean event completed.

13. Practice Tasks

1. Time a kernel using start/stop events and compare it with correct end-to-end CPU timing.
2. Build a producer stream and consumer stream connected by one event.
3. Poll an event with `cudaEventQuery` while doing useful CPU work.
4. Implement a ring of three buffers with one completion event per slot.
5. Profile a device-wide barrier version and an event-dependent version.

14. Final Cheat Sheet

Item	Must remember
Core definition	A completion/timestamp marker in a stream.
Why it matters	Fine-grained dependencies, completion, and timing.
Most asked	Event synchronize vs stream wait event.
Key comparison	GPU event time vs CPU wall time.
Main trap	Event record is asynchronous.
One-line answer	"Record in the producer, wait in the consumer, and leave unrelated work running."

Overlapping Compute and Memory Transfer

1. Overview

Overlapping compute and memory transfer means a GPU kernel executes on streaming multiprocessors while a DMA copy engine simultaneously moves data between host and device, typically for a different chunk of data.

Overlap hides transfer time behind computation. If a transfer takes 4 ms and a kernel takes 7 ms, a well-formed steady-state pipeline may approach roughly 7 ms per chunk instead of 11 ms, subject to startup, drain, bandwidth contention, and hardware limits. This pattern is used in streaming analytics, inference, video, simulations, and out-of-core processing. Interviewers ask for the exact conditions required, not merely "use streams."

2. Core Idea

Think of washing and drying clothes. With one batch, washing then drying takes the sum of both times. With multiple batches and separate machines, batch 1 can dry while batch 2 washes. The first and last batches still pay pipeline fill and drain costs.

```
for (int i = 0; i < nStreams; ++i) {
    cudaMemcpyAsync(d[i], h_in[i], bytes, cudaMemcpyHostToDevice, s[i]);
    process<<<grid, block, 0, s[i]>>>(d[i]);
    cudaMemcpyAsync(h_out[i], d[i], bytes, cudaMemcpyDeviceToHost, s[i]);
}
```

Required reasoning:

1. Divide the input into independent chunks.
2. Keep each chunk's H2D → kernel → D2H chain in one stream.
3. Put different chunks in different streams.
4. Use pinned host buffers.
5. Ensure the device supports concurrent copy and execution.
6. Keep buffers distinct until their stream completes.
7. Profile the timeline and tune chunk size/count.

3. Important Subtopics

Subtopic	Meaning and importance	Example	Common interview angle
Device capability	<code>asyncEngineCount</code> and related properties describe copy/compute support.	One or more copy engines.	Capability permits overlap; it does not guarantee it.
Chunking	Large work is divided into pipeline units.	Four 32 MB segments.	Too small adds overhead; too large leaves little overlap.
Pinned memory	Host buffers support direct DMA.	<code>cudaMallocHost</code> .	Pageable memory often breaks expected overlap.
Stream placement	Independent chunks use different nondefault streams.	Stream per ring-buffer slot.	Same-stream copy and kernel are ordered, not overlapped.
Double/triple buffering	Multiple storage slots prevent producer/consumer races.	Camera frames A/B/C.	Use an event to mark each slot reusable.
Pipeline fill/drain	First and last stages cannot be fully hidden.	Startup H2D and final D2H.	Speedup improves over many chunks.
Resource contention	Compute and copy can compete for device memory bandwidth.	Bandwidth-bound kernel plus D2H.	Timeline overlap may not produce additive throughput.

4. Real-World Example

An out-of-core matrix transformation processes a dataset larger than GPU memory. Three slots are used: one transfers the next tile to the GPU, one computes the current tile, and one copies the previous result back. Events prevent the CPU from refilling a pinned slot until its copy has completed.

For many equally sized chunks, a simplified steady-state model is:

```

serial time/chunk    ≈ T_H2D + T_kernel + T_D2H
pipelined throughput ≈ max(T_H2D path, T_kernel, T_D2H path)

```

This is an optimistic model; shared links, one bidirectional engine, and memory contention change it.

5. Diagrams / Mental Models

```

Time --->
Stream 0: [H2D 0][Kernel 0-----][D2H 0]
Stream 1:      [H2D 1][Kernel 1-----][D2H 1]
Stream 2:      [H2D 2][Kernel 2-----][D2H 2]

Engines:
H2D copy: [0]      [1]      [2]
SMs:      [K0-----][K1-----][K2-----]
D2H copy:      [0]      [1]      [2]

```

The profiler's engine view is more trustworthy than assuming this ideal schedule from source code.

6. Common Interview Questions

1. **What conditions are required for copy/compute overlap?** Supported hardware, asynchronous copies, pinned host memory, independent work in different streams, and no serializing dependency/default-stream effect. Mistake: answering only "use two streams."
2. **Why can't a copy and following kernel in one stream overlap?** Same-stream order makes the kernel wait for the copy. Mistake: assuming copy engine and SM independence overrides dependencies.
3. **Why split data into chunks?** It creates independent pipeline units so one chunk can transfer while another computes. Mistake: using one giant transfer and expecting full overlap.
4. **How do you choose chunk size?** Benchmark a range; balance launch/copy overhead, pipeline occupancy, memory capacity, and compute duration. Mistake: giving one universal size.
5. **What is double buffering?** Two buffer slots alternate ownership so producer and consumer operate on different batches. Mistake: reusing a slot without a completion event.
6. **What speedup is theoretically possible?** Serial stage sum can approach the longest steady-state stage, not zero transfer cost. Mistake: adding bandwidths or promising 2× universally.
7. **How do you verify overlap?** Inspect an Nsight Systems timeline and measure end-to-end throughput after warm-up. Mistake: inferring it from asynchronous calls alone.
8. **Can H2D, kernel, and D2H all overlap?** Possibly on hardware with suitable engines and independent resources; directions and paths are architecture-dependent. Mistake: assuming every GPU supports three-way overlap.
9. **Why might overlap make each kernel slower?** Kernel and copy may contend for device memory bandwidth, caches, interconnect, or power budget. Mistake: judging only visual concurrency.
10. **Does overlap reduce latency or throughput?** Primarily improves steady-state throughput; a single item may see similar or greater latency due to queuing. Mistake: treating throughput and latency as identical.

7. Deep-Dive Questions

1. **Derive pipeline time for N chunks.** In an ideal balanced multi-engine pipeline, total time is approximately fill time + $(N-1) \times \text{bottleneck_stage}$ + drain time. Exact scheduling depends on engine topology.
2. **Why can breadth-first submission affect overlap?** Enqueuing all H2D copies, then all kernels, then all D2H copies can create different engine queues and dependencies than submitting a complete per-chunk chain. The best ordering can vary by architecture; profile it.
3. **How does a bandwidth-bound kernel affect overlap?** DMA and kernel both demand memory bandwidth, so physical concurrency may only divide the same bottleneck and yield little throughput gain.
4. **How would you handle variable-sized chunks?** Use bounded slots, per-slot events, and load balancing; avoid letting one long chunk block all subsequent reuse.
5. **When is overlap not worth implementing?** When transfers are already negligible, the workload is too small, pageable input ownership cannot change, or added buffering exceeds the measured gain.

8. Comparison Tables

Serial staging	Overlapped pipeline
H2D then kernel then D2H	Different chunks occupy stages concurrently
Time approaches stage sum	Throughput approaches slowest stage
One buffer can suffice	Multiple slots usually required
Simpler lifetimes	Requires explicit ownership/completion
Double buffering	Triple buffering
Two in-flight ownership states	Can represent H2D, compute, and D2H slots simultaneously
Less memory	More scheduling flexibility
Often sufficient	Useful for three-stage pipelines or jitter
Easier reasoning	More events and memory footprint

9. Common Mistakes

- Using pageable host memory.
- Putting all operations in the legacy default stream.
- Reusing one buffer across streams without dependencies.
- Assuming device capability guarantees application overlap.
- Choosing tiny chunks that are dominated by launch/copy overhead.
- Looking only at kernel duration instead of end-to-end throughput.
- Ignoring fill/drain overhead for small numbers of chunks.

10. Edge Cases / Special Cases

- A device with one copy engine may serialize H2D and D2H even while overlapping either with compute.
- Integrated GPUs may share physical memory, changing the value of explicit transfers.

- Small transfers may not visibly overlap due to fixed overhead and timeline resolution.
- A kernel occupying the memory system can make concurrent transfers slower.
- NUMA topology, PCIe switches, and multiple GPUs can make the host link the bottleneck.

11. How to Explain in Interview

"I overlap transfer and compute by chunking independent data, using pinned host buffers, and submitting each chunk's H2D–kernel–D2H chain to a separate stream. The copy engines can then work alongside SMs. I verify hardware capability and profiler overlap because streams only expose the opportunity."

12. Quick Revision Notes

- Needs pinned memory, separate streams, independent chunks, and hardware support.
- Same-stream stages remain ordered.
- Steady-state throughput tends toward the slowest stage.
- Fill and drain remain visible.
- More overlap can create bandwidth contention.
- Profile engine timelines.
- Interview trap: overlap improves throughput more directly than single-item latency.

13. Practice Tasks

1. Implement serial and two-stream chunked vector pipelines; compare throughput.
2. Sweep chunk sizes and plot effective bandwidth/end-to-end time.
3. Add a third stream and identify the point of diminishing returns.
4. Replace pinned memory with pageable memory and inspect the timeline.
5. Estimate ideal pipeline time from measured stage times, then explain the gap from reality.

14. Final Cheat Sheet

Item	Must remember
Core definition	DMA moves one chunk while SMs compute another.
Why it matters	Hides transfer time and improves throughput.
Most asked	Conditions required for overlap.
Key comparison	Serial sum vs pipelined bottleneck stage.
Main trap	Streams alone do not create overlap.
One-line answer	"Chunk + pin + separate streams + independent engines, then verify in the profiler."

Concurrent Kernels

1. Overview

Concurrent kernels are kernels from different streams whose execution intervals overlap on the GPU. The scheduler may place thread blocks from multiple kernels on the same or different SMs when resources are

available.

Concurrent kernels can improve utilization when one kernel does not fill the GPU, has insufficient parallelism, or leaves complementary resources unused. They appear in multi-request inference, task-parallel simulations, independent model components, and small-kernel workloads. Interviewers ask about concurrent-kernel limits to see whether you understand occupancy, block scheduling, and why different streams are necessary but insufficient.

2. Core Idea

Imagine a hotel where each kernel sends groups of guests (thread blocks), and SM resources are rooms, beds, and staff (registers, shared memory, warp slots). If kernel A fills every room, kernel B waits even if it has a separate reservation line. If A uses only half the hotel, blocks from B may fit.

```
kernelA<<<gridA, blockA, 0, s1>>>(a);
kernelB<<<gridB, blockB, 0, s2>>>(b);
```

For overlap: the launches must be in independent streams; no event/default-stream dependency may serialize them; both grids must have runnable work at the same time; and A must leave resources/capacity for B. CUDA schedules blocks—not arbitrary fractions of a block—and does not promise a particular interleaving.

3. Important Subtopics

Subtopic	Meaning and importance	Example	Common interview angle
Concurrent-kernel capability	Device property indicates support.	<code>concurrentKernels</code> .	Support is not a guarantee for every pair.
Occupancy/resources	Registers, shared memory, threads, blocks, and warp slots constrain co-residency.	A high-register kernel blocks B.	Occupancy is a resource calculation, not performance itself.
Grid size	A small grid may leave SMs idle.	8 blocks on 80 SMs.	Concurrency can fill unused SMs.
Block granularity	Scheduler assigns whole blocks to SMs.	Residual resources too small for B's block.	Complementary percentages do not always fit.
Priorities	Higher-priority pending work is favored.	Latency request vs background kernel.	Typically no guaranteed mid-block preemption.
Hyper-Q/work queues	Hardware supports multiple work queues,	Multiple streams feeding work.	Architecture affects available concurrency.

Subtopic	Meaning and importance	Example	Common interview angle
	reducing false dependencies.		
Resource contention	Concurrent kernels share compute, caches, and memory bandwidth.	Two memory-bound kernels.	Overlap may reduce each kernel's speed.

4. Real-World Example

An inference service batches large requests but also receives small latency-sensitive requests. A large kernel in one stream may leave some resources, allowing a small kernel in a high-priority stream to co-reside. However, if the large kernel launches enough blocks to occupy every SM with high register/shared-memory usage, the small kernel may wait until blocks retire.

5. Diagrams / Mental Models

Possible co-residency on one SM:

```
+-----+
| Kernel A block | Kernel A block | Kernel B block |
+-----+
limited by threads + warps + registers + shared memory + block slots
```

No room case:

```
+-----+
| Kernel A block | Kernel A block | A resources... |
+-----+
Kernel B queued despite being in another stream.
```

6. Common Interview Questions

- What are concurrent kernels?** Kernels whose device execution intervals overlap, normally launched in different streams. Mistake: calling back-to-back asynchronous launches concurrent without evidence.
- Can kernels in the same stream run concurrently?** Same-stream execution order prevents later work from overtaking earlier work. Use different streams for concurrency. Mistake: assuming asynchronous host launches are enough.
- Do different streams guarantee concurrent kernels?** No. Dependencies, grid size, resource use, and hardware scheduling determine overlap. Mistake: answering yes.
- What resources limit co-residency?** Registers, shared memory, threads, warps, block slots, SM count, and sometimes other architectural limits. Mistake: mentioning only thread count.
- When is concurrent execution most useful?** Small or underutilizing kernels, independent requests, or workloads with complementary bottlenecks. Mistake: overlapping two kernels that already saturate the same resource.
- Can two kernels share an SM?** On capable devices, blocks from different kernels may co-reside if resources fit. Mistake: claiming each SM belongs exclusively to one kernel.

7. **Does 100% occupancy prevent concurrency?** Often it leaves no residency capacity, but occupancy metrics and resource granularity require exact analysis. Also high occupancy is not synonymous with full performance. Mistake: using one percentage as proof.
8. **How do stream priorities affect kernels?** They influence scheduling of pending work but do not guarantee immediate preemption of running blocks. Mistake: promising real-time behavior.
9. **How do you verify concurrent kernels?** Use a profiler timeline plus resource/occupancy analysis and end-to-end throughput measurements. Mistake: relying on source launch order.
10. **Can concurrent kernels be slower than serial execution?** Yes, due to contention for compute pipelines, cache, memory bandwidth, and scheduling overhead. Mistake: treating visual overlap as speedup.

7. Deep-Dive Questions

1. **Why can lowering registers per thread enable concurrency but hurt performance?** It may permit more resident blocks, yet cause register spilling to local memory. The end-to-end tradeoff must be measured.
2. **What if kernel A has fewer blocks than SMs?** Unused SMs can accept blocks from kernel B, making concurrency straightforward if no dependency exists.
3. **Can two memory-bound kernels improve throughput together?** Usually little if one already saturates bandwidth; they may only share bandwidth and increase latency. Complementary compute/memory behavior has a better chance.
4. **How does block size influence co-residency?** Resources are allocated at block/warp granularity. Smaller blocks may fit residual capacity, though too-small blocks can reduce efficiency.
5. **Would kernel fusion be better than concurrency?** Fusion can remove launch/global-memory boundaries and share data, but may increase registers, reduce occupancy, or couple independent work. Compare measured costs and correctness constraints.

8. Comparison Tables

Sequential kernels	Concurrent kernels
One execution interval after another	Intervals overlap
Predictable resource ownership	Resources are shared/co-resident
May leave idle SM capacity	Can fill underutilized capacity
No cross-kernel contention	Possible cache/bandwidth/compute contention
Concurrent kernels	Kernel fusion
Keeps separate launches and code	Combines stages into one kernel
Flexible independent scheduling	Can eliminate intermediate traffic/launches
Useful for unrelated tasks	Useful for tightly coupled stages
May contend for resources	May increase per-thread resource use

9. Common Mistakes

- Assuming separate streams force concurrent execution.

- Ignoring register and shared-memory allocation.
- Launching both kernels into the same or legacy default stream.
- Optimizing occupancy number instead of application throughput.
- Using priority as if it were hard preemption.
- Overlapping two bandwidth-saturating kernels and expecting double throughput.
- Confusing CPU launch overlap with GPU execution overlap.

10. Edge Cases / Special Cases

- A kernel with a tiny grid may overlap because it leaves entire SMs idle.
- A persistent kernel can prevent other work by intentionally occupying most SM resources.
- Cooperative launches impose special residency/synchronization constraints and can limit concurrency.
- Dynamic parallelism and device-side launches add scheduling constraints beyond ordinary host streams.
- Multi-Instance GPU and MPS change resource isolation/sharing, but do not remove per-partition limits.

11. How to Explain in Interview

“Concurrent kernels are kernels from independent streams that overlap on the GPU. The scheduler can co-reside their blocks only if SM resources such as registers, shared memory, warp slots, and block slots fit. I use concurrency mainly to fill underutilized capacity and confirm benefit with a profiler because resource contention can erase the gain.”

12. Quick Revision Notes

- Different streams are necessary for ordinary concurrency.
- Hardware capability and available resources decide actual overlap.
- Scheduling occurs at block granularity.
- Priorities are scheduling hints, not guaranteed preemption.
- Best candidate: underutilizing or complementary kernels.
- Profiler overlap is not enough; measure throughput.
- Interview trap: high occupancy is neither guaranteed performance nor a full concurrency model.

13. Practice Tasks

1. Launch two small independent kernels in separate streams and inspect their overlap.
2. Increase grid size until concurrency disappears; explain why.
3. Vary dynamic shared memory to change co-residency.
4. Compare concurrent launches against fused and sequential versions.
5. Use an occupancy calculator/API to predict resource limits, then validate them in Nsight Systems/Compute.

14. Final Cheat Sheet

Item	Must remember
Core definition	Execution intervals of independent kernels overlap.
Why it matters	Fills otherwise unused GPU capacity.

Item	Must remember
Most asked	Why separate streams may still serialize.
Key comparison	Concurrent launches vs kernel fusion.
Main trap	Capability and streams do not guarantee co-residency.
One-line answer	"Kernels overlap only when dependencies permit and whole blocks fit available GPU resources."

Stream Synchronization

1. Overview

Stream synchronization establishes when queued work must complete or when one operation must wait for another. CUDA provides scopes ranging from a device-wide host barrier to a single event dependency between two streams.

Synchronization matters for correctness: without it, a host may read incomplete results or streams may race on shared data. It also matters for performance because overly broad barriers serialize independent work. Real systems use synchronization at buffer ownership changes, pipeline stage boundaries, API handoffs, and result consumption. Interviewers ask candidates to choose the narrowest correct primitive and explain its host/device effects.

2. Core Idea

Think of synchronization as traffic control. A device-wide barrier closes every road until all cars arrive. A stream wait event places one traffic light at one intersection; unrelated roads remain open.

```
producer<<<g, b, 0, s1>>>(d);
cudaEventRecord(ready, s1);
cudaStreamWaitEvent(s2, ready, 0);
consumer<<<g, b, 0, s2>>>(d);
```

Step by step: producer writes `d`; event marks producer completion; consumer stream receives a dependency; consumer runs only after the event; host does not need to block; unrelated streams continue. Synchronization should reflect the real data dependency, not merely make the program "feel safe."

3. Important Subtopics

Subtopic	What it means and why it matters	Example	Common interview angle
Device synchronization	Host waits for preceding work across the	<code>cudaDeviceSynchronize</code> .	Simple but broad and expensive in a

Subtopic	What it means and why it matters	Example	Common interview angle
	device/context scope.		pipeline.
Stream synchronization	Host waits for prior work in one stream.	<code>cudaStreamSynchronize(s).</code>	Narrower than device sync but still blocks host.
Event synchronization	Host waits through one recorded marker.	<code>cudaEventSynchronize(e).</code>	Useful for one buffer or phase completion.
Event query	Host checks completion without waiting.	<code>cudaEventQuery(e).</code>	Enables polling plus useful CPU work.
Cross-stream wait	A stream waits on an event without blocking host.	<code>cudaStreamWaitEvent.</code>	Preferred producer-consumer dependency.
Same-stream ordering	Earlier operations complete before dependent later operations execute.	H2D → kernel.	An extra barrier is unnecessary inside the chain.
Implicit synchronization	Some default-stream, memory, and API behavior adds ordering.	Legacy null-stream interaction.	Hidden serialization is a performance trap.
Memory lifetime/visibility	Storage cannot be reused until all accessing work completes.	Ring-buffer slot event.	Completion is an ownership boundary.

4. Real-World Example

An inference server maintains eight request slots. Each slot has device buffers, a stream, and a completion event. Before refilling slot **i**, the CPU queries or waits only for slot **i**'s event. It never calls `cudaDeviceSynchronize()` in the request loop, so unrelated requests continue running.

```
slot 0 event complete? yes -> reuse slot 0
slot 1 event complete? no  -> leave it; process other CPU work
slot 2 event complete? yes -> return result and reuse
```

5. Diagrams / Mental Models

```
Broad barrier:
s0: [A-----] -> stop
s1: [B----]     -> stop      Host waits for all
s2: [C-----] -> stop
```

```

Fine-grained event dependency:
s0: [producer] -> (E)
      \
s1: [unrelated] ---- wait(E) -> [consumer]
s2: [fully unrelated work continues-----]

```

Synchronization scope ladder:

```

same-stream order < cross-stream event < event host wait
                  < stream host wait < device-wide host wait

```

Choose based on the actual dependency, not merely this apparent cost ordering.

6. Common Interview Questions

1. **What does `cudaDeviceSynchronize()` do?** It blocks the calling host thread until preceding device work in its scope completes and reports asynchronous errors. Mistake: using it after every kernel by default.
2. **What does `cudaStreamSynchronize(s)` do?** It blocks the host until previously submitted work in stream `s` completes. Mistake: saying it synchronizes all streams.
3. **How can two streams synchronize without blocking the CPU?** Record an event in the producer and enqueue `cudaStreamWaitEvent` in the consumer. Mistake: calling `cudaEventSynchronize` first, which blocks the host unnecessarily.
4. **Do consecutive operations in one stream need explicit synchronization?** No for their device execution order; the stream orders them. Host access still needs a completion boundary. Mistake: adding a barrier between every operation.
5. **What is the narrowest way to wait for one output?** Usually synchronize/query the event marking that output or its stream, depending on ownership design. Mistake: device-wide sync.
6. **Does synchronization make two racing writes correct?** Only if it orders them as intended. A barrier after both writes does not retroactively resolve their race. Mistake: synchronizing too late.
7. **What is implicit synchronization?** Ordering introduced by APIs/default-stream behavior without an explicit wait call. Mistake: looking only for functions containing "Synchronize."
8. **How does the host know a D2H async result is ready?** The copy's stream/event must complete before the host reads the destination. Mistake: reading after `cudaMemcpyAsync` returns.
9. **Why is excessive synchronization slow?** It drains queues, prevents overlap, idles host/device resources, and reduces batching. Mistake: blaming only function-call overhead.
10. **Can synchronization surface earlier kernel errors?** Yes, completion APIs can report asynchronous execution errors from prior work. Mistake: treating every sync error as caused by the synchronize function itself.

7. Deep-Dive Questions

1. **What is the difference between execution dependency and host blocking?** An execution dependency orders device work; host blocking stops a CPU thread. `cudaStreamWaitEvent` gives the former without the latter.

2. **Can a device-wide barrier guarantee another host thread has submitted its work?** No. It waits for work already in the relevant CUDA scope; host-thread coordination may still be required around submission.
3. **How do you safely recycle ring-buffer slots?** Record a completion event after the last use of each slot and reuse only after that event completes. One event per slot keeps ownership explicit.
4. **Why can default-stream mixing cause surprising waits?** Legacy null-stream semantics impose broad ordering with blocking streams, connecting work that source code may appear to keep independent.
5. **How would you debug a synchronization race?** Reduce to the shared allocation, map every read/write to streams, draw happens-before edges, use sanitizer race tools where applicable, and add the minimal missing event dependency.

8. Comparison Tables

Primitive	Who waits?	Scope	Typical use
Same-stream order	Later device operation	One stream	Sequential pipeline stages
<code>cudaStreamWaitEvent</code>	One device stream	Work after the wait	Cross-stream producer-consumer
<code>cudaEventQuery</code>	Nobody	One event marker	Nonblocking completion check
<code>cudaEventSynchronize</code>	Host thread	Through one event marker	Wait for one buffer/result
<code>cudaStreamSynchronize</code>	Host thread	Prior work in one stream	Finish one pipeline/request
<code>cudaDeviceSynchronize</code>	Host thread	Prior device work	Global phase/debug boundary
Correct synchronization		Over-synchronization	
Encodes actual data dependencies		Adds unrelated ordering	
Preserves independent overlap		Drains pipelines	
Uses events/streams where possible		Uses device-wide barriers habitually	
Clear buffer ownership		Often hides weak ownership design	

9. Common Mistakes

- Calling `cudaDeviceSynchronize()` after every launch.
- Assuming different streams order shared-memory accesses automatically.
- Synchronizing after a race instead of ordering the conflicting accesses.
- Freeing or overwriting memory before all stream users finish.
- Confusing host wait functions with device-side dependencies.
- Missing legacy default-stream implicit ordering.
- Ignoring error codes returned at synchronization points.

10. Edge Cases / Special Cases

- Blocking host waits and spin/yield behavior can depend on runtime scheduling flags.
- Stream capture restricts some synchronization APIs because a graph must represent valid dependencies.
- Host callbacks/functions occur at stream-defined points but should not perform prohibited CUDA calls or long blocking work.
- Events and streams are associated with CUDA contexts/devices; cross-device usage has explicit restrictions.
- Memory pools and stream-ordered allocation/free APIs tie allocation lifetime to stream dependency semantics.

11. How to Explain in Interview

“CUDA synchronization establishes happens-before relationships for device work or blocks the host until completion. Same-stream order handles local chains; events connect streams; event or stream synchronization waits for specific host-visible results; device synchronization is a broad phase barrier. I choose the narrowest primitive that protects the data.”

12. Quick Revision Notes

- Same stream already provides device ordering.
- `cudaStreamWaitEvent`: device waits, host continues.
- `cudaEventSynchronize`: host waits for one marker.
- `cudaStreamSynchronize`: host waits for one stream.
- `cudaDeviceSynchronize`: broad host barrier.
- Synchronization also exposes async errors.
- Interview trap: a barrier after conflicting work does not fix the earlier race.

13. Practice Tasks

1. Replace a device-wide barrier with a producer event and consumer stream wait.
2. Build a ring buffer with per-slot completion events.
3. Create a two-stream data race, draw its missing happens-before edge, and fix it.
4. Compare throughput using per-request stream sync versus one final device sync.
5. Run Compute Sanitizer tools on a deliberately mis-synchronized sample and interpret the report.

14. Final Cheat Sheet

Item	Must remember
Core definition	Establish completion or ordering across host/device work.
Why it matters	Prevents races while preserving safe concurrency.
Most asked	Device sync vs stream sync vs event wait.
Key comparison	Host-blocking wait vs device-side dependency.
Main trap	Broad synchronization destroys overlap and may only hide races.

Item	Must remember
------	---------------

One-line answer	"Synchronize the dependency, not the entire GPU."
-----------------	---